

IaC-Guard-V: A Verification Framework for LLM-Generated Infrastructure-as-Code Repairs

Lokesh Chauhan^[0009–0004–1544–6424]

Independent Researcher, Dallas, TX, USA
lokesh0186@gmail.com

Abstract. Infrastructure-as-Code (IaC) misconfigurations are a leading cause of cloud security incidents, yet the trustworthiness of LLM-generated repairs remains underexplored, especially for cross-technology, multi-gate verification. We present IaC-Guard-V, a verification framework that evaluates AI-generated IaC repairs through four gates: syntactic validity, target-issue resolution, regression safety, and patch minimality. We construct a benchmark of 70 misconfigured Terraform and Kubernetes artifacts spanning 70 unique scanner rules across eight violation classes, and evaluate three repair strategies across three LLM families in 630 runs. Results show that while all models achieve 100% syntactic validity, only 32–50% pass full verification under single-shot prompting. Verification-guided iterative repair achieves 68–92%, and an open-weight model with verification outperforms the strongest commercial model without it at less than one-tenth the cost. Structured prompting consistently degrades Terraform repair quality, a counterintuitive finding that challenges common assumptions.

Keywords: Infrastructure-as-Code · Large language models · Automated program repair · Verification · Software reliability · Terraform

1 Introduction

Infrastructure-as-Code (IaC) has become the standard mechanism for provisioning cloud infrastructure, enabling teams to define servers, networks, and security policies through machine-readable files such as Terraform HCL and Kubernetes YAML. However, IaC misconfigurations remain a leading cause of cloud security incidents: 65% of organizations experienced a cloud-related security incident in the past year, with misconfigurations ranking among the top causes [2,10]. Static analysis tools such as Checkov, KICS, and Terrascan detect these misconfigurations, but remediation remains manual and error-prone.

This work was conducted independently and does not relate to the author’s employment. No employer resources, data, or confidential information were used.

Accepted as a regular paper at QRS 2026. This is the author’s accepted manuscript. The final authenticated version will appear in the Springer LNCS proceedings.

Large Language Models (LLMs) are increasingly used as automated repair agents. In the broader domain of automated program repair (APR), LLM-based systems fix hundreds of real-world bugs at costs below \$1 per fix [19], and Detect–Repair–Verify (DRV) workflows iteratively refine patches using test feedback [4]. Practitioners are now applying LLMs to IaC repair, deploying generated patches with minimal review.

This practice is risky. While LLM-generated IaC repairs may appear syntactically correct, their trustworthiness (whether they resolve the target issue without introducing new vulnerabilities) remains underexplored for IaC specifically. General-purpose DRV workflows rely on unit tests for verification, which does not transfer to IaC: infrastructure artifacts lack executable test suites, security scanners may produce inconsistent results, and provider-specific rules fragment the verification process.

Motivating example. Consider a Terraform configuration with two CloudWatch log groups (one with a 3-day retention and one with no retention at all), which triggers Checkov rule CKV_AWS_338 (“CloudWatch log groups should retain logs for at least one year”). When prompted to fix the misconfiguration, Claude Sonnet correctly raises the retention on the first log group from 3 days to 365, but leaves the second resource untouched. The fix is syntactically valid, but the rule continues to fail because the second log group still has no retention configured. Without verification, a practitioner reviewing the patch could plausibly accept it. Our verification harness catches this at gate V2, and the iterative repair loop produces a complete fix on the second attempt; we walk through the full evaluation in Section 7.

We present **IaC-Guard-V**, a verification-centered framework evaluating LLM-generated IaC repairs through four verification dimensions: three binary gates, namely (V1) syntactic validity, (V2) target-issue resolution, and (V3) regression safety, together with one informational metric, (V4) patch minimality. We define the *verified-fix rate* (VFR) and construct a benchmark of 70 misconfigured artifacts (50 Terraform, 20 Kubernetes) spanning 70 unique scanner rules and eight violation classes.

We make the following contributions:

1. **Framework.** IaC-Guard-V, with four verification gates tailored to IaC challenges, including a verification-guided iterative repair loop that feeds scanner feedback back to the LLM.
2. **Benchmark.** A multi-technology benchmark of 70 misconfigured IaC artifacts with stratified selection across 70 unique rules and eight violation classes.
3. **Empirical study.** A comprehensive evaluation of 630 runs across three LLM families (Claude Opus 4.6, Claude Sonnet 4.6, Llama 4 Maverick), three repair strategies, and two technologies.
4. **Key findings.** (a) Only 32–50% of syntactically valid repairs pass full verification. (b) Verification-guided repair achieves 68–92% VFR. (c) An open-weight model with verification outperforms the strongest commercial model without it at less than one-tenth the cost. (d) Structured prompting con-

sistently degrades Terraform repair quality. (e) Technology complexity and model capability jointly determine trustworthiness.

All artifacts are released at <https://github.com/lokes0186/iac-guard-v>.

2 Background and Related Work

We organize related work into three areas and position our study in Table 1.

2.1 LLM-Based Automated Program Repair

LLMs have transformed APR. Xia et al. [18] demonstrated practical LLM-based repair. Zhang et al. [22] provide the first systematic review of 189 LLM-based APR papers. Yang et al. [20] propose a taxonomy grouping 62 systems into four paradigms.

Among individual systems, ChatRepair [19] pioneered conversation-driven repair, fixing 162/337 Defects4J bugs at \$0.42 each. RepairLLaMA [15] showed that fine-tuned open-source models match commercial ones. RepairAgent [1] introduced autonomous agent-based repair. FLAMES [7] reduces memory by 83% via semantic-guided search.

Most relevant, Cheng [4] introduced DRV for securing LLM-generated code, and Chen et al. [3] showed LLMs can self-debug. While DRV establishes the iterative verification paradigm we build upon, it targets general-purpose code with unit tests, an approach that does not transfer to IaC.

2.2 IaC-Specific Repair and Generation

TerraFormer [5] combines fine-tuning with verifier-guided RL for Terraform generation, but focuses on NL-to-IaC rather than repair. InfraFix [13] proposes SMT-based repair across Ansible, Puppet, Chef, and Terraform (95.7% success on 254K scenarios), but uses constraint solving rather than LLMs. GenKubeSec [8] fine-tunes a local LLM for Kubernetes repair (0.990 precision on 277K configs). LLMSecConfig [21] combines Checkov with LLMs and RAG (94.3% success on 1K configs). IaCGen [23] introduces a deployability benchmark achieving 54.6–91.6% within 10 iterations. Palavalli and Santolucito [9] find that feedback effectiveness plateaus after ~ 5 iterations, corroborating our convergence analysis.

2.3 IaC Benchmarks and Defect Taxonomies

Rahman et al. [11] identified seven categories of security smells in 1,726 Puppet scripts. Sharma et al. [14] cataloged 24 configuration smells. Saavedra and Ferreira [12] proposed GLITCH for cross-technology smell detection. Verdet et al. [16] conducted the first large-scale study of Terraform security using Checkov across 812 projects. IaC-Eval [6] shows GPT-4 achieves only 19.36% pass@1 on Terraform, far below Python benchmarks.

Gap. No prior work combines LLM-based repair, multi-dimensional verification (syntax, resolution, regression, minimality), and cross-technology IaC evaluation. IaC-Guard-V addresses this gap.

Table 1: Positioning of this study against related work.

Study	Focus	IaC LLM	Verif.	Repair	Bench.
DRV [4]	General code	No	Yes	Tests	Yes
ChatRepair [19]	Conv. APR	No	Yes	Tests	Yes
TerraFormer [5]	IaC gen + RL	Yes	Yes	Policy	No
InfraFix [13]	IaC repair (SMT)	Yes	No	Partial	Yes
GenKubeSec [8]	K8s repair	Yes	Yes	Detect	Yes
LLMSecConfig [21]	Container	Yes	Yes	Checkov	Yes
IaC-Eval [6]	IaC gen bench	Yes	Yes	Formal	No
This study	IaC repair verif.	Yes	Yes	4 gates	Yes

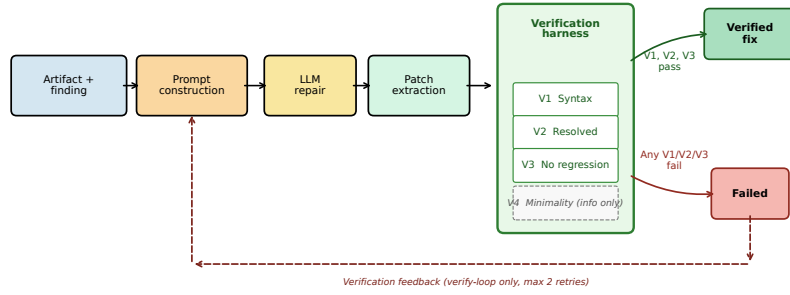


Fig. 1: IaC-Guard-V pipeline. A misconfigured artifact is repaired by an LLM and evaluated by three binary verification gates plus an informational minimality metric. In verify-loop mode, failed verification results are fed back for up to two retries.

3 IaC-Guard-V Framework

Figure 1 illustrates the end-to-end pipeline.

3.1 Verification Gates

Given an original artifact A , a repaired artifact $A' = f(A, v)$ produced by an LLM for scanner finding v , we define a *verified fix* as:

$$VF(A, A', v) = V_1(A') \wedge V_2(A', v) \wedge V_3(A, A') \quad (1)$$

Patch minimality (V_4) is measured but does not gate the verdict.

V1: Syntactic Validity. The repaired artifact must be parseable by the target technology’s parser. We use Checkov’s built-in parser: if Checkov can scan the file and produce structured output, the artifact is syntactically valid.

V2: Target-Issue Resolution. The specific finding v must no longer appear in Checkov’s output for A' . We re-run Checkov and verify that the target rule ID is absent from the failed-checks list.

V3: Regression Safety. The repair must not introduce new findings. We compare failed rule IDs on A' against the baseline on A . Any new rule constitutes a regression.

V4: Patch Minimality. We measure changed lines via unified diff. Unlike V1–V3, minimality is informational: large patches suggest the LLM rewrote unrelated sections.

3.2 Verified-Fix Rate

The primary metric is:

$$VFR = \frac{|\{i : V_1(A'_i) \wedge V_2(A'_i, v_i) \wedge V_3(A_i, A'_i)\}|}{N} \quad (2)$$

3.3 Repair Strategies

Plain prompting. The LLM receives the artifact and a generic fix instruction. No rule-specific information is provided.

Structured prompting. The LLM receives the artifact plus rule ID, description, violation class, affected resource, and line range. It must return a JSON object with the fixed artifact, summary, and change details.

Verification-guided iterative repair (verify-loop). The first attempt uses structured prompting. If verification fails, the harness constructs a retry prompt containing the original artifact, the failed attempt, and gate-specific feedback. Up to two retries are allowed (three total attempts).

3.4 Verification Procedure

Algorithm 1 presents the complete verification procedure. All gates are fully automated; no manual judgment is involved at any point.

The verification harness executes Checkov in quiet mode with JSON output (`checkov -f <file> --output json --quiet --compact`). For V2, we check the target rule ID against the failed-checks list. For V3, we compute the set difference between repaired and baseline failed rules. For V4, we compute a unified diff and report the ratio of changed lines to original file length.

Automated validation. For the primary VFR metric, cross-annotator agreement is not applicable because V1–V3 are deterministic comparisons of scanner outputs. Manual inspection is used only for the secondary failure categorization in RQ5; the classification criteria and raw logs are released for independent replication.

3.5 Implementation

IaC-Guard-V is implemented in Python (~1,200 lines) with four modules: prompt constructor, Bedrock API caller (supporting Anthropic and Meta model families), verification harness, and experiment runner. The framework is model-agnostic by design: any LLM accessible via API can be substituted without modifying the verification logic.

Algorithm 1: Verification-Guided Iterative Repair

Input: Artifact A , finding v , LLM M , max retries $k = 2$
Output: Verified fix A' or FAIL

```

1  $B \leftarrow \text{RUNCHECKOV}(A)$  ; // Baseline findings
2  $A' \leftarrow M.\text{REPAIR}(A, v, \text{prompt}=\text{structured})$ ;
3 for  $i \leftarrow 0$  to  $k$  do
4    $V_1 \leftarrow \text{CANPARSE}(A')$ ;
5    $O \leftarrow \text{RUNCHECKOV}(A')$ ;
6    $V_2 \leftarrow (v \notin O.\text{failed\_rules})$ ;
7    $V_3 \leftarrow (O.\text{failed\_rules} \setminus B.\text{failed\_rules} = \emptyset)$ ;
8    $V_4 \leftarrow \text{DIFFRATIO}(A, A')$ ;
9   if  $V_1 \wedge V_2 \wedge V_3$  then
10     $\text{return } A'$  ; // Verified fix
11   if  $i < k$  then
12      $fb \leftarrow \text{FORMATFEEDBACK}(V_1, V_2, V_3, O)$ ;
13      $A' \leftarrow M.\text{RETRY}(A, A', fb)$ ;
14 return FAIL;

```

4 Benchmark Construction

4.1 Source and Selection

We source items from Checkov’s official test suite (v3.2.517), containing IaC artifacts written to validate specific scanner rules. For Terraform, we scanned 287 AWS-specific directories with 1,081 violations across 369 rules and selected 50 items using stratified sampling (one per unique rule, over-sampling minority classes). For Kubernetes, we scanned 98 directories with 2,400 violations across 215 rules and selected 20 items.

4.2 Violation Taxonomy

Table 2 summarizes the benchmark. Items are classified into eight classes based on security impact.

4.3 Baseline Establishment

For each item, we run Checkov and record all failed rules as the baseline. This confirms the target rule is present and provides the reference for regression detection (V3).

4.4 Benchmark Validity and False Positives

Because the benchmark is sourced from Checkov’s maintained rule tests, the target findings serve as scanner-grounded positives for the evaluated Checkov

Table 2: Benchmark composition by violation class.

Violation Class	Terraform	Kubernetes
Missing encryption	12	2
Over-permissive access	8	6
Network hardening	8	3
Public exposure	6	0
Weak observability	6	1
Insecure defaults	5	3
Missing runtime safety	0	4
Other	5	1
Total	50	20

Each item maps to a unique Checkov rule (70 total).

version. This design does not estimate Checkov’s production false-positive rate. For the regression gate, we conservatively treat any new Checkov finding after repair as a potential regression; some such findings could be scanner-specific false positives. Multi-scanner verification with Checkov, KICS, and Terrascan is left for future work, and all raw scanner outputs are released for independent inspection.

5 Experimental Setup

5.1 Models

We evaluate three LLMs via AWS Bedrock: Claude Opus 4.6 and Sonnet 4.6 (Anthropic, commercial) and Llama 4 Maverick (Meta, open-weight, 17B active parameters, mixture-of-experts). This spans two commercial models of different capability and one open-weight model.

5.2 Configuration

All models use temperature 0 for deterministic outputs. Maximum output: 4,096 tokens. Verify-loop: up to two retries. Checkov v3.2.517.

5.3 Evaluation Protocol

$70 \times 3 \times 3 = 630$ experimental runs. For each run: raw response, extracted patch, all gate results, token counts, and latency. For verify-loop: per-attempt results for convergence analysis.

5.4 Statistical Methods

95% bootstrap CIs (1,000 resamples, fixed seed). Cochran’s Q for overall method differences. Six pre-specified pairwise comparisons using McNemar’s exact test with Bonferroni correction ($\alpha = 0.05/6 = 0.0083$). Cliff’s delta for effect sizes.

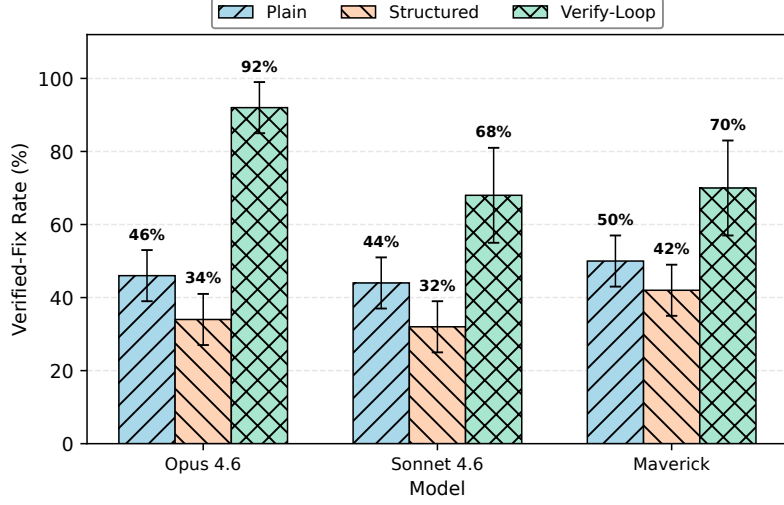


Fig. 2: Verified-fix rate by model and strategy (Terraform, $n=50$). Error bars: 95% bootstrap CIs.

6 Results

6.1 RQ1: Effectiveness of Verification-Guided Repair

Motivation. We evaluate whether verification-guided iterative repair produces more trustworthy fixes than single-shot prompting on the Terraform benchmark (50 items).

Table 3 and Figure 2 present the results. Three findings emerge.

First, all models achieve 100% syntactic validity, but only 34–52% of syntactically valid repairs resolve the target issue under single-shot prompting.

Second, verify-loop substantially outperforms both baselines: Opus 92% (CI: 84–98%), Maverick 70% (56–82%), Sonnet 68% (54–80%), vs. 32–50% for baselines.

Third, structured prompting consistently underperforms plain prompting (32–42% vs. 44–50%) and introduces regressions in 4–10% of cases.

Cochran’s Q rejects equal method performance ($p < 0.001$). McNemar’s with Bonferroni ($\alpha = 0.05/6 \approx 0.0083$): verify-loop significantly outperforms structured for all three models (Opus $p < 0.001$, $\delta = 0.58$; Sonnet $p < 0.001$, $\delta = 0.36$; Maverick $p < 0.001$, $\delta = 0.28$). Verify-loop also significantly outperforms plain for all three models (Opus $p < 0.001$, $\delta = 0.46$; Sonnet $p = 0.0075$, $\delta = 0.24$; Maverick $p = 0.0063$, $\delta = 0.20$). All six pre-specified pairwise comparisons remain significant under Bonferroni correction.

Table 3: Terraform repair results ($n=50$ per cell). V1=Syntax, V2=Resolved, V3=No Regression (%).

Model	Method	V1	V2	V3	VFR	95% CI
Opus	Plain	100	46	100	46	32–60
	Structured	100	34	96	34	22–46
	Verify-loop	100	92	98	92	84–98
Sonnet	Plain	100	44	100	44	30–58
	Structured	100	34	90	32	20–44
	Verify-loop	100	76	92	68	54–80
Maverick	Plain	100	52	88	50	36–64
	Structured	100	42	100	42	28–56
	Verify-loop	100	74	94	70	56–82

Table 4: Verify-loop VFR by violation class (Terraform, aggregated across 3 models).

Violation Class	Plain	Verify-loop	Δ
Missing encryption	83	97	+14
Public exposure	17	94	+78
Weak observability	56	83	+28
Over-permissive access	46	71	+25
Network hardening	33	67	+33
Other	20	53	+33
Insecure defaults	33	47	+13

All values are percentages; Δ is computed from unrounded rates.

6.2 RQ2: Variation by Violation Class

Table 4 presents verify-loop VFR by class. Fix rates range from 47% (insecure defaults) to 97% (missing encryption). Public-exposure shows the largest improvement from verify-loop (+78pp), suggesting verification feedback is most valuable for violations where initial attempts are incomplete.

Per-model disaggregation reveals extreme variation: Opus achieves 100% on missing encryption and insecure defaults while Sonnet achieves 0% on the “other” class. To investigate this divergence we inspected the raw repaired artifacts for all five items. All five (BM-0004 through BM-0008) derive from a single multi-resource Checkov test file (`example_ALBDesyncMode/main.tf`) containing both intentionally-passing (suffix `.pass`) and intentionally-failing (suffix `.fail`) variants of three load-balancer resource types: `aws_lb`, `aws_alb`, and `aws_elb`. Sonnet’s verify-loop consistently applies fixes only to the `.pass` variants and leaves the `.fail` variants unchanged across all three retry attempts. Because Checkov reports the target rule at the file level (not per-resource), the rule remains in the failed-checks list and V2 fails. Opus and Maverick ap-

Table 5: Kubernetes repair results ($n=20$ per cell, %).

Model	Method	V1	V2	V3	VFR
Opus	Plain/Struct./V-loop	100	100	100	100
Sonnet	Plain/Struct./V-loop	100	100	100	100
Maverick	Plain	100	70	55	45
	Structured	95	95	95	95
	Verify-loop	100	100	100	100

Table 6: Cost-effectiveness (all 70 items). Structured omitted (underperforms plain).

Model	Method	VFR	Avg Tok.	Latency	\$/fix
Maverick	Plain	48.6	1,027	1.7s	.001
Maverick	Verify-loop	78.6	3,163	5.0s	.002
Sonnet	Plain	60.0	1,217	5.1s	.018
Sonnet	Verify-loop	77.1	4,188	17.6s	.044
Opus	Plain	61.4	1,214	6.4s	.029
Opus	Verify-loop	94.3	3,320	15.2s	.047

ply the fix to both variants and achieve V2 resolution on 4 of 5 items; the fifth (BM-0006/CKV_AWS_127) requires a valid ACM certificate ARN that no model produces. This pattern is consistent with the multi-resource isolation effect we observe in BM-0276 and BM-0449 (Section 6.5), and we treat all such cases consistently in our threats-to-validity sensitivity analysis (Section 9).

6.3 RQ3: Cross-Technology Generalization

Table 5 presents Kubernetes results. Both commercial models achieve 100% VFR across all methods, while Maverick achieves 45% with plain prompting before verify-loop rescues it to 100%.

Structured prompting *improves* Maverick’s K8s performance (45%→95%), opposite to Terraform. We attribute the technology gap to: (1) Kubernetes YAML has flat, well-documented schemas; (2) Terraform encodes provider-specific semantics across hundreds of resource types; (3) Terraform files often contain multiple intentionally misconfigured resources.

6.4 RQ4: Cost-Effectiveness

Table 6 presents cost metrics. Maverick with verify-loop achieves 78.6% VFR at \$0.002/fix and 5.0s latency. Opus without verification achieves 61.4% at \$0.029/fix, more than 10× more expensive per successful repair while delivering 17 percentage points lower VFR.

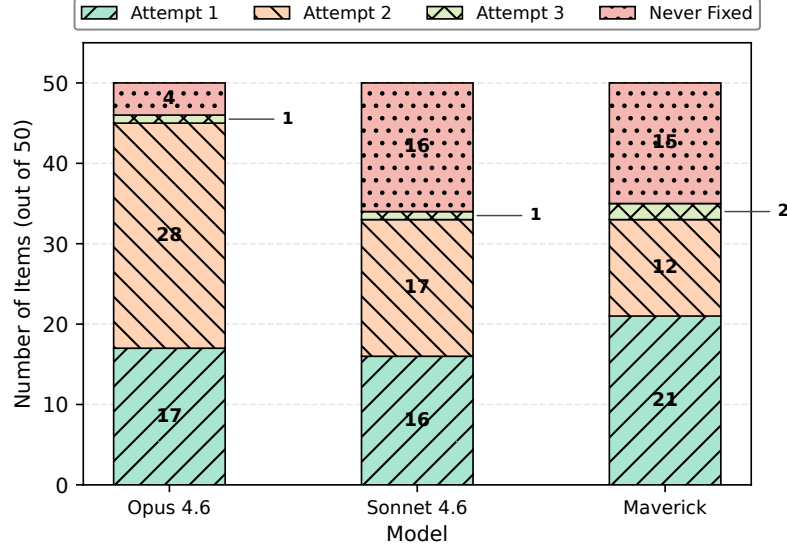


Fig. 3: Verify-loop convergence: fixes by attempt number (Terraform). The second retry (attempt 3) adds only approximately 2 percentage points over attempt 2.

Patch minimality. Structured prompting produces the most minimal patches (median diff ratio 0.064–0.075 across the three models) but sacrifices correctness (32–42% VFR on Terraform). Verify-loop optimizes for correctness (68–92% VFR) at slightly larger patch sizes (median 0.106–0.145). Maverick’s verify-loop patches (median 0.141) are substantially smaller than its plain patches (median 0.433), indicating that verification feedback helps focus the open-weight model’s repairs.

6.5 RQ5: Failure Analysis

Of 50 Terraform items under verify-loop: 30 fixed by all models, 4 by none, 16 model-dependent. The 35 individual failures fall into three categories: (1) two harness limitations where Checkov cannot isolate the target resource; (2) one graph-based rule (CKV2_AWS_33) checking cross-resource dependencies; (3) remaining model limitations.

Convergence. Figure 3 shows most fixes occur in attempts 1–2; the second retry (attempt 3) adds only 2pp on average over attempt 2, corroborating Palavalli and Santolucito’s observation that feedback plateaus after ~5 iterations [9].

Cross-model agreement. 30/50 items (60%) fixed by all models, 4 (8%) by none. Seven items fixed exclusively by Opus confirm its unique capability on harder items. Zero items fixed exclusively by Sonnet or Maverick.

7 Concrete Example: Verification in Action

To illustrate the framework’s operation, we present a complete walkthrough of benchmark item BM-0075 (CKV_AWS_338: “Ensure CloudWatch log groups retain logs for at least 1 year”). The original artifact contains two CloudWatch log groups in the same file, each violating the rule for a different reason: the first specifies a retention shorter than the required 365 days, and the second specifies no retention at all.

Listing 1.1: Original artifact (BM-0075): both resources violate CKV_AWS_338. The resource labels `pass` and `fail` are inherited from the upstream Checkov fixture and should not be interpreted as this study’s pass/fail verdict labels.

```
resource "aws_cloudwatch_log_group" "pass" {
  retention_in_days = 3
}

resource "aws_cloudwatch_log_group" "fail" {}
```

Attempt 1 (Sonnet, structured prompt). The model returns:

Listing 1.2: Sonnet’s first attempt: partial fix, with one resource updated and the other untouched.

```
resource "aws_cloudwatch_log_group" "pass" {
  retention_in_days = 365
}

resource "aws_cloudwatch_log_group" "fail" {}
```

Gate evaluation:

- V1 (Syntax): **PASS**. Checkov parses the file successfully.
- V2 (Resolution): **FAIL**. CKV_AWS_338 still appears in Checkov output because `aws_cloudwatch_log_group.fail` retains no `retention_in_days` attribute.
- V3 (Regression): **PASS**. No new findings introduced.
- V4 (Minimality): diff ratio = 0.107 (computed over the full artifact in released logs).
- **Verdict: NOT a verified fix.**

Feedback to LLM (illustrative). The harness constructs a retry prompt summarising each gate’s verdict together with the relevant Checkov output. For this case the feedback conveys: gate V2 (target-issue resolution) failed because CKV_AWS_338 is still present in Checkov’s failed-checks list; the rule requires every CloudWatch log group to set `retention_in_days` to at least 365; the modification to the first resource alone does not satisfy the rule for the second resource.

Attempt 2 (retry with feedback). The model corrects its approach:

Listing 1.3: Sonnet’s second attempt: complete fix, both resources now satisfy the rule.

```
resource "aws_cloudwatch_log_group" "pass" {
  retention_in_days = 365
}

resource "aws_cloudwatch_log_group" "fail" {
  retention_in_days = 365
}
```

Gate evaluation:

- V1 (Syntax): **PASS**. Valid HCL.
- V2 (Resolution): **PASS**. CKV_AWS_338 no longer in Checkov output.
- V3 (Regression): **PASS**. No new findings.
- V4 (Minimality): diff ratio = 0.393 (computed over the full artifact in released logs).
- **Verdict: VERIFIED FIX** (achieved on attempt 2).

This example demonstrates two key properties: (1) syntactic validity is a poor proxy for correctness, since attempt 1 produces valid HCL but the rule remains violated by the untouched second resource; and (2) verification feedback enables targeted self-correction by identifying *which* gate failed and *why*.

8 Discussion

Verification matters more than model capability. Maverick with verify-loop (70% VFR, \$0.002/fix) outperforms Opus without it (46%, \$0.029/fix). Even within the same family, Opus plain (46%) barely exceeds Sonnet plain (44%), while Opus verify-loop (92%) substantially outperforms both. For budget-constrained deployments, verify-loop enables smaller models to approach larger models’ quality.

The structured prompting paradox. Contrary to common assumptions [17], structured prompting consistently underperforms on Terraform. Two mechanisms: (1) the JSON schema forces serializing the entire file as a string value, increasing escaping errors; (2) providing the specific rule causes narrow focus, missing other instances in multi-resource files. This reverses on Kubernetes (Maverick: 45%→95%), where single-resource files eliminate the multi-resource problem.

Retry convergence. Most fixes occur on attempts 1–2. Three items where Sonnet’s plain succeeds but verify-loop fails reveal a “retry confusion” effect (6%), but the net gain remains strongly positive (+12 items for Sonnet).

On baselines. Our baselines, plain and structured single-shot prompting, represent the most common practitioner workflow. Stronger baselines such as RAG-augmented repair [21] or fine-tuned models [5] are orthogonal to our verification contribution. IaC-Guard-V is a verification framework, not a repair

technique; demonstrating that even simple strategies benefit substantially from verification feedback is itself a contribution.

Practical recommendations. (1) For Kubernetes with commercial models, plain prompting + post-hoc verification suffices (100% VFR). (2) For Terraform or open-weight models, verify-loop is essential. (3) Two retries is the cost-effective sweet spot. (4) Integrate the verification harness into CI/CD pipelines as a mandatory gate.

9 Threats to Validity

Internal. Our harness relies on Checkov as the sole scanner. The harness applies a file-level rule check for V2: a rule is considered resolved only if it no longer appears in Checkov’s failed-checks list for the entire file. In multi-resource files where the same rule is violated by several resources, this means a repair must address every violating resource, not just the one nominally targeted. This affects three classes of items in our benchmark: (a) BM-0276 and BM-0449, where the target rule is also triggered by an unrelated resource that no model can plausibly modify; (b) BM-0040, which involves a graph-based rule (CKV2_AWS_33) that checks cross-resource dependencies and which no model resolves; (c) BM-0004 through BM-0008, which derive from a single Checkov test file containing both intentionally-passing and intentionally-failing variants of three load-balancer resource types. As a sensitivity analysis on the Terraform benchmark, excluding (a) raises Opus verify-loop from 92.0% to 95.8%; further excluding (b) yields 97.9%; further excluding (c) yields 97.7% on the remaining 43 items. For Sonnet, excluding (a) raises verify-loop from 68.0% to 70.8%; further excluding (b) and (c) raises it to 79.1%. The relative ordering of methods (verify-loop $\gg$ structured, verify-loop $\geq$ plain) is preserved under all such exclusions, and the Cochran’s Q rejection of equal method performance ($p < 0.001$) holds in every subset.

Construct. VFR treats V1–V3 as binary gates. V4 is informational only: correct but overly large repairs pass. Human evaluation of intent preservation would complement automated metrics.

External. Our 70 items cover 70 unique rules across eight classes and two technologies. The 630 runs provide sufficient power (Cochran’s Q $p < 0.001$). Items are sourced from Checkov’s test suite rather than production repos. Results are specific to April 2026 model versions. Extending to CloudFormation, Pulumi, and multi-scanner verification is planned.

Conclusion. Temperature 0 ensures deterministic outputs. All comparisons use McNemar’s exact test with Bonferroni correction over the six pre-specified comparisons ($\alpha = 0.05/6 \approx 0.0083$). All six comparisons are significant at the corrected threshold; the smallest gap is for Sonnet verify-loop vs. plain ($p = 0.0075$, $\delta = 0.24$), still meeting the corrected threshold by ~ 0.001 . Bootstrap CIs use 1,000 resamples with a fixed random seed for reproducibility.

10 Conclusion and Future Work

We presented IaC-Guard-V, a verification framework for LLM-generated IaC repairs with four verification gates. Our evaluation of 630 runs demonstrates that syntactic validity is a poor proxy for correctness (100% vs. 32–50% full verification), verification-guided repair achieves 68–92% VFR, and verification infrastructure matters more than model capability.

Three directions merit investigation. First, multi-scanner consensus (Checkov + KICS + Terrascan) to reduce scanner-specific biases. Second, a hybrid prompting strategy exploiting complementary strengths: plain prompting for the initial attempt (higher first-attempt VFR) combined with structured prompting for retries (focused corrections), as our data shows these strategies have complementary failure modes. Third, scaling the benchmark to additional technologies and production repositories.

All artifacts: <https://github.com/lokes0186/iac-guard-v>.

Acknowledgments. Portions of this work used AI-assisted tools only for grammar checking and LaTeX formatting support. All framework design, implementation, benchmark construction, experimental execution, statistical analysis, failure categorization, and interpretations are solely the author’s work.

Disclosure of Interests. The author has no competing interests to declare that are relevant to the content of this article.

References

1. Bouzenia, I., Devanbu, P., Pradel, M.: RepairAgent: An autonomous, LLM-based agent for program repair. arXiv:2403.17134 (2024). <https://doi.org/10.48550/arXiv.2403.17134>
2. Check Point Software Technologies: 2025 cloud security report. <https://www.checkpoint.com/downloads/products/2025-cloud-security-report.pdf> (2025)
3. Chen, X., Lin, M., Schärli, N., Zhou, D.: Teaching large language models to self-debug. In: Proc. ICLR (2024)
4. Cheng, C.: Detect–repair–verify for LLM-generated code: A multi-language, multi-granularity empirical study. arXiv:2603.23633 (2026). <https://doi.org/10.48550/arXiv.2603.23633>
5. Jana, P., Davidson, S., Bhasker, B., Kan, A., Deoras, A., Callot, L.: TerraFormer: Automated infrastructure-as-code with LLMs fine-tuned via policy-guided verifier feedback. In: Proc. ICSE-SEIP (2026)
6. Kon, P., et al.: IaC-Eval: A code generation benchmark for cloud infrastructure-as-code programs. In: NeurIPS Datasets and Benchmarks (2024). <https://doi.org/10.52202/079017-4273>
7. Le-Cong, T., Le, B., Murray, T.: Memory-efficient large language models for program repair with semantic-guided patch generation. In: Proc. ICSE (2026)
8. Malul, E., Meidan, Y., Mimran, D., Elovici, Y., Shabtai, A.: GenKubeSec: LLM-based Kubernetes misconfiguration detection, localization, reasoning, and remediation. In: Proc. ACSAC (2024)

9. Palavalli, M.A., Santolucito, M.: Using a feedback loop for LLM-based infrastructure as code generation. *arXiv:2411.19043* (2024). <https://doi.org/10.48550/arXiv.2411.19043>
10. Palo Alto Networks, Unit 42: Cloud threat report, volume 7: Navigating the expanding attack surface. <https://www.paloaltonetworks.com/resources/research/unit-42-cloud-threat-report-volume-7> (2024)
11. Rahman, A., Parnin, C., Williams, L.: The seven sins: Security smells in infrastructure as code scripts. In: *Proc. ICSE* (2019). <https://doi.org/10.1109/ICSE.2019.00033>
12. Saavedra, N., Ferreira, J.a.F.: GLITCH: Automated polyglot security smell detection in infrastructure as code. In: *Proc. ASE* (2022). <https://doi.org/10.1145/3551349.3556945>
13. Saavedra, N., Ferreira, J.a.F., Mendes, A.: InfraFix: Technology-agnostic repair of infrastructure as code. In: *Proc. ISSTA* (2025). <https://doi.org/10.1145/3713081.3731735>
14. Sharma, T., Fragkoulis, M., Spinellis, D.: Does your configuration code smell? In: *Proc. MSR* (2016). <https://doi.org/10.1145/2901739.2901761>
15. Silva, A., Fang, S., Monperrus, M.: RepairLLaMA: Efficient representations and fine-tuned adapters for program repair. *IEEE Trans. Softw. Eng.* (2025). <https://doi.org/10.1109/TSE.2025.3581062>
16. Verdet, A., Hamdaqa, M., Da Silva, L., Khomh, F.: Assessing the adoption of security policies by developers in Terraform across different cloud providers. *Empirical Softw. Eng.* (2025). <https://doi.org/10.1007/s10664-024-10610-0>
17. Wei, J., et al.: Chain-of-thought prompting elicits reasoning in large language models. In: *Proc. NeurIPS* (2022)
18. Xia, C.S., Wei, Y., Zhang, L.: Automated program repair in the era of large pre-trained language models. In: *Proc. ICSE* (2023). <https://doi.org/10.1109/ICSE48619.2023.00129>
19. Xia, C.S., Zhang, L.: Automated program repair via conversation: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT. In: *Proc. ISSTA* (2024). <https://doi.org/10.1145/3650212.3680323>
20. Yang, B., Cai, Z., Liu, F., Le, B., Zhang, L., Bissyandé, T.F., Liu, Y., Tian, H.: A survey of LLM-based automated program repair: Taxonomies, design paradigms, and applications. *arXiv:2506.23749* (2025). <https://doi.org/10.48550/arXiv.2506.23749>
21. Ye, Z., Le, T.H.M., Babar, M.A.: LLMSecConfig: An LLM-based approach for fixing software container misconfigurations. *arXiv:2502.02009* (2025). <https://doi.org/10.48550/arXiv.2502.02009>
22. Zhang, Q., Fang, C., Xie, Y., Ma, Y., Sun, W., Yang, Y., Chen, Z.: A systematic literature review on large language models for automated program repair. *ACM Trans. Softw. Eng. Methodol.* (2026). <https://doi.org/10.1145/3799693>
23. Zhang, T., Pan, S., Zhang, Z., Xing, Z., Sun, X.: Deployability-centric infrastructure-as-code generation: Fail, learn, refine, and succeed through LLM-empowered DevOps simulation. *arXiv:2506.05623* (2025). <https://doi.org/10.48550/arXiv.2506.05623>